AutoBayes against brms: speed on ten models

Benchmark run of 17 October 2026, prepared for Andrew

17 October 2026

The answer. Yes, the speedup holds, and it holds for the Tier C and D models as well as the four from the status report. Fitting each model from scratch (compiling and sampling), AutoBayes was 2.3 to 12.7 times faster than brms, with a median of 5.9 times. Counting sampling alone, it was 1.2 to 7.9 times faster (median 3.9). Its effective sample size per second of sampling was higher on nine of the ten models. On the same fits brms's post-processing functions were 2 to 7 times slower than AutoBayes's. The two libraries' posteriors agree. The four models that were also in the status report reproduced its numbers within about 10% for brms; for AutoBayes they reproduced within the same margin except B1, which was faster this time (5.3 s against 8.1 s).

1 What was run

Each model was fitted by both libraries with the same formula, family and explicit priors (no defaults), on the same simulated data (`examples/brms_parity/benchmark_data.py`, fixed seeds, every numeric predictor standardised). The sampler settings were also the same: four chains in parallel, 1000 warm-up and 1000 kept draws each, target acceptance 0.9.

- **brms** 2.23.0 with rstan 2.32.7. Compilation (`brm(chains = 0)`) and sampling (`update(recompile = FALSE)`) were timed separately.
- **AutoBayes** sampled with NumPyro's NUTS on the CPU (`chain_method="parallel"`). Its time is the model build plus the fit, and the fit includes JAX's compilation. Its "sampling" time is the fit minus a one-draw fit of the same model, which estimates the compilation.
- The machine was an Apple M1 Pro laptop in ordinary use, with desktop applications open and a Time Machine backup running when the run started. Each model was run once, brms first and then AutoBayes, never both at once.

2 Time to a posterior

Most of the end-to-end difference on small models is brms's compilation. Turning a brms model into a C++ program and compiling it took 26 to 36 seconds per model; AutoBayes's JAX compilation took 2 to 4. On B10, for example, brms spent 26.8 of its 30.0 seconds compiling and 3.2 sampling, while AutoBayes took 2.4 seconds in all. Sampling itself was also faster in AutoBayes, but by less, and by very different amounts:

- **Large gains (5–8×)** came on the models with many rows or an ordinal likelihood: B1–B3 and B10. On B2 (20,000 rows) sampling took 16.5 seconds against brms's 87.0.
- **Small gains (1.2–1.4×)** came on B6, B8 and B9. On B8 both samplers finish in about 2–3 seconds, so fixed overheads dominate. B6 and B9 evaluate log-gamma functions and their derivatives on every row (the negative binomial and the beta-binomial). That is a plausible cause of the small gain there, but I have not profiled it.

	model	rows	brms (s)			AutoBayes (s)	AutoBayes faster by	
			compile	sample	total	total	total	sampling
B1	Bernoulli, `(1 + x | g)`	2,000	36.2	5.8	42.0	5.3	7.9	5.4
B2	Gaussian, `(1 + x | g)`	20,000	35.5	87.0	122.5	29.6	4.1	5.3
B3	cumulative, five categories	1,500	27.0	33.4	60.4	7.1	8.5	7.9
B4	zero-inflated Poisson	2,000	26.2	15.0	41.2	5.9	7.0	4.1
B5	Weibull	2,000	27.1	6.4	33.5	6.0	5.6	3.3
B6	zero-inflated neg. binomial	2,000	26.6	16.4	43.0	14.7	2.9	1.4
B7	sratio, five categories	1,500	26.1	17.6	43.6	7.1	6.2	3.7
B8	Gaussian, horseshoe on 20	400	26.6	3.0	29.6	5.3	5.6	1.3
B9	beta-binomial, trials	2,000	27.7	24.4	52.1	22.3	2.3	1.2
B10	Gaussian, `(1 | mm(g1, g2))`	1,000	26.8	3.2	30.0	2.4	12.7	6.6

Table 1: Wall times, one run each. B1, B2, B3 and B5–B7 and B9 have a varying intercept (and B1, B2 a slope) on 30 to 200 groups.

3 Is the faster sampler sampling as well?

Raw speed means little if the draws are worse. Table 2 reports the smallest bulk effective sample size over the parameters both libraries report, computed by the same ArviZ estimator for both, and that number divided by the sampling time.

	min. bulk ESS		ESS per second of sampling		divergent		max $\lvert z \rvert$
	AutoBayes	brms	AutoBayes	brms	AutoBayes	brms	
B1	1264	1017	1184	176	0	0	1.88
B2	125	144	8	2	0	0	1.22
B3	530	558	126	17	0	0	0.41
B4	386	414	106	28	0	0	1.46
B5	201	287	105	45	0	0	1.02
B6	1079	1245	90	76	0	0	0.70
B7	1087	1195	230	68	0	0	1.86
B8	1916	2257	841	752	9	6	2.71
B9	185	264	9	11	0	0	1.43
B10	533	499	1093	154	0	0	0.82

Table 2: Sampling efficiency and agreement. z is the difference of the two posterior means over their combined Monte Carlo standard error, for each parameter both report; the largest over the parameters is shown.

- The ESS per second favours AutoBayes on nine models. Its smallest ESS is sometimes lower (B5: 201 against 287; B9: 185 against 264), but it gets there faster. On B5, 201 effective draws in 1.9 seconds is about 105 a second, against brms's 287 in 6.4 seconds, about 45.
- The exception is B9, the beta-binomial: 9 effective draws a second against brms's 11. On sampling efficiency alone, brms is ahead here. In every model the slowest parameter for both libraries is the population intercept (a threshold for the ordinal models, a coefficient on B8). That is probably the familiar slow mixing of the intercept against the mean of the varying intercepts, and it is the same in both libraries.
- The posteriors agree: every parameter's mean differs by less than 2.8 combined Monte Carlo

standard errors, and by less than 1.9 except on the horseshoe regression (2.71). That is the agreement expected from two independent samplers.
- Both libraries hit divergent transitions only on the horseshoe regression (9 and 6 of 4,000). They are characteristic of that prior's funnel, not of either library.

4 Post-processing

Table 3 times brms's functions on each fit against AutoBayes's: the median of three calls after a first one.

	`posterior_epred`	`log_lik`	`loo`	`waic`	`bayes_R2`	`hypothesis`	`cond._effects`
B1	0.11 / 0.37	0.51 / 0.75	2.10 / 3.41	0.62 / 2.66	0.16 / 0.89	0.00 / 0.00	0.02 / 0.17
B2	0.65 / 1.97	2.66 / 5.81	18.62 / 38.94	3.97 / 20.30	0.94 / 3.68	0.00 / 0.00	0.02 / 0.15
B3	1.77 / 1.75	0.56 / 1.27	1.69 / 3.58	0.64 / 2.51	1.91 / 2.37	*ref.* / 0.00	*ref.* / 0.25
B4	0.13 / 0.35	0.34 / 1.50	1.93 / 3.93	0.45 / 2.77	0.16 / 0.65	0.00 / 0.00	0.02 / 0.13
B5	0.11 / 0.36	0.18 / 0.98	1.91 / 3.57	0.30 / 2.13	0.15 / 0.65	0.00 / 0.00	0.02 / 0.14
B6	0.13 / 0.35	0.98 / 1.77	2.59 / 4.25	1.08 / 2.96	0.16 / 0.64	0.00 / 0.00	0.02 / 0.13
B7	1.29 / 9.63	1.19 / 1.98	2.35 / 4.27	1.32 / 3.11	1.47 / 10.47	*ref.* / 0.00	*ref.* / 0.79
B8	0.03 / 0.06	0.03 / 0.10	0.31 / 0.60	0.05 / 0.36	0.04 / 0.13	0.00 / 0.00	0.03 / 0.22
B9	0.14 / 0.41	1.45 / *n/a*	3.06 / *n/a*	1.56 / *n/a*	0.18 / 0.70	0.00 / 0.00	*ref.* / 0.14
B10	0.05 / 0.09	0.07 / 0.36	0.79 / 1.81	0.13 / 1.00	0.06 / 0.20	0.00 / 0.00	0.02 / 0.14

Table 3: Seconds, AutoBayes / brms. *ref.*: AutoBayes refuses the function for this model, by design (divergence D99: the ordinal families' thresholds, and the trials a beta-binomial grid would need). *n/a*: brms needs the R package `extraDistr` for the beta-binomial's `log_lik`, and it is not installed here.

The median ratios over the models where both ran were: `posterior_epred` 2.8×, `log_lik` 2.3×, `loo` 2.0×, `waic` 5.1×, `bayes_R2` 4.0× and `conditional_effects` 6.9×. `hypothesis` takes a few milliseconds at most in both. The largest absolute gap is PSIS-LOO on B2: 18.6 seconds against 38.9, for 20,000 observations and 4,000 draws.

5 What these numbers do not show

- **One run each.** The run was on a laptop in normal use, so a single number may be off by 10–30%. The four models repeated from the status report came out within that range of their earlier times.
- **Repeated fits.** brms compiles once per model and can refit new data without recompiling (`update(recompile = FALSE)`). For many fits of one model, its 26–36 seconds are paid once and only the "sampling" column matters.
- **Other Stan setups.** cmdstanr, which runs a newer Stan and may sample faster than rstan, is not installed here and was not tested. Neither was brms's within-chain threading.
- **GPUs.** The CPU only: no GPU result is claimed here.
- **Scope.** Ten models on simulated data from each model's own family. A model that is hard to sample (a funnel, weak identification) is a different test.

6 Reproducing

```
python examples/brms_parity/benchmark_data.py DATA
```

```
Rscript examples/brms_parity/benchmark_brms.R DATA OUT
python examples/brms_parity/benchmark_autobayes.py DATA OUT
python examples/brms_parity/compare.py OUT
```

The summary of this run is `examples/brms_parity/results/comparison-2026-10-17.json`; the draws are not committed.